Building and Analyzing Monolingual Transformer Language Models in Two Indian Languages

Final Report

Erva Rishitha (2025201027)

Checkpoints and dataset links:
OneDrive Archive – Primary Data Storage(Datasets and Phase1)
Phase 1 – OCR & Reference Data
Phase 2 – Pretraining Checkpoints
Phase 3 – Reasoning Data & Checkpoints

Abstract

In this project, I have built two independent monolingual language models from scratch, I have choose telugu for high-resource language(Model H) and Nepali for low-resource language(Model L). I kept the two models completely separate, using different corpora, tokenizers, vocabularies, and model weights for each language. For both languages, I collected and cleaned roughly 500M tokens, with more than 20% of the data coming from sources that I collected manually using web crawling and OCR. I trained separate SentencePiece BPE tokenizers with a vocabulary size of 12,000 and implemented a decoder-only Transformer with about 25.3M parameters using basic PyTorch components. The attention mechanism, positional embeddings, causal masking, and other Transformer components were implemented directly rather than using a pretrained Transformer model. After pretraining, I also finetuned the two models separately on synthetic comparative reasoning datasets in their respective languages. One of the main observations from the experiments was that language modelling performance and reasoning performance did not follow the same trend. Model L achieved a lower test perplexity of 33.09 compared with 38.70 for Model H. However, on the reasoning task, Model H achieved an exact match accuracy of 64.69%, while Model L achieved 52.74%. In other words, the model with the lower perplexity did not perform better on the reasoning task. This difference made it useful to look beyond perplexity and consider factors such as the training corpus, tokenization, and the amount and type of data available for each language.

1 Project Setup

Model H (higher-resource): Telugu: I picked Telugu because it has strong coverage in AI4Bharat's Sangraha corpus, a complete Wikimedia ecosystem, and a lot of active news sites I could crawl, so the 500M-token target felt achievable without depending on a single source. It's also my mother tongue, which turned out to matter more than I expected, because I could actually read the model's outputs and tell whether they were good or just plausible-looking.

Model L (lower-resource): Nepali: Nepali is on the assignment's permitted low-resource list, and Sangraha itself flags its Nepali subset as low-resource. It's written in Devanagari rather than Telugu's abugida, which gave me a useful script contrast for the tokenizer analysis later on. The two models never touch each other's data, weights, or tokenizer at any point.

2 Phase 1: Corpus and Tokenizer

2.1 Collection and cleaning

Every corpus is split into two tiers, tracked separately end to end. The **downloaded tier** comes streamed from AI4Bharat Sangraha. The **manual tier** is everything I gathered myself: news and government sitemap/RSS crawls with per-domain rate limiting, Wikimedia dumps and live API passes, BFS crawls over literature sites, Internet Archive sweeps, and GPU OCR of scanned textbooks and archived PDFs using PyMuPDF and EasyOCR.

Everything checkpoints: Crawls write `crawled_urls.json` and `failed_urls.json` so an interrupted run can resume, and the OCR notebooks only commit a book once every page succeeds, so a session that dies partway through discards its partial output instead of leaving a half-written book sitting in the corpus. This mattered more than I expected going in, since the crawls ran across local, Colab, and Kaggle sessions, and none of those are guaranteed to stay alive for as long as a crawl needs.

Cleaning is identical for both languages: HTML unescape, NFC normalization, stripping out the log and progress-bar artifacts that leaked in from early versions of the crawler, URL and wiki-markup removal, then a per-source filter on minimum length and minimum script-character ratio. OCR text gets a stricter script threshold (0.60–0.75) than clean web text (0.45), because OCR misreads tend to produce stray Latin characters. Anything under 30 words after cleaning gets dropped.

Deduplication runs in two passes: SHA-256 exact matching, then MinHash+LSH near-duplicate removal (128 permutations, 5-word shingles, Jaccard 0.85). In both passes the manual tier is streamed first through a single shared index, so if a downloaded document turns out to be a near-duplicate of something in the manual tier, it's the downloaded copy that gets dropped.

Table 1: Final corpus statistics, after cleaning, deduplication, and splitting.

	Telugu (H)	Nepali (L)
Documents	983,814	1,186,541
Total tokens	**547.5M**	**526.6M**
Manual tokens	**28.12%**	**20.22%**
Downloaded tokens	71.88%	79.78%
Fertility (chars/token)	4.1275	4.4793
UNK rate (val / test)	0.00%	0.00%
Train tokens	542.0M	521.3M
Val tokens	2.80M	2.68M
Test tokens	2.64M	2.59M

Table 2: Token share by source.

Source	Telugu	Nepali
Sangraha (downloaded)	70.95%	79.78%
News websites	26.29%	15.44%
Government portals	—	3.25%
Archive OCR	1.05%	0.01%
OCR textbooks	0.92%	1.02%
Literature websites	0.56%	0.31%
Wikimedia APIs	0.23%	0.19%

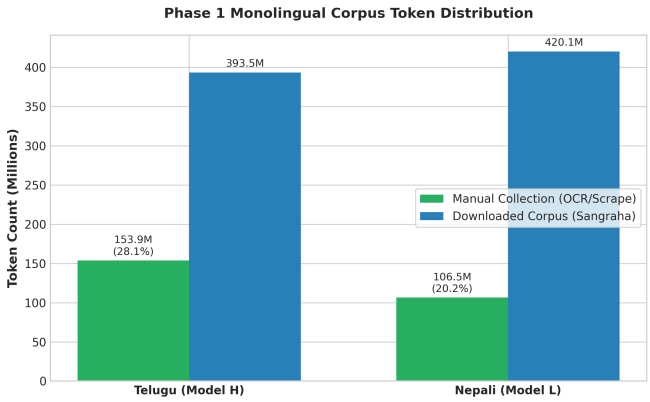


Figure 1: Manual vs. downloaded token share for both corpora

Both corpora clear the requirement, but not by the same margin. Telugu's manual data is around 28.12%. Nepali's maual data is around 20.22%. Splits are produced by shuffling and splitting each tier independently (99/0.5/0.5, seed 42) and then concatenating, so the manual share stays almost constant across splits instead of clustering into just one of them.

2.2 Tokenizer

I trained SentencePiece BPE tokenizer per language, with byte fallback and `nmt_nfkc` normalization. Byte fallback has one important consequence: the UNK rate is exactly 0.00% at every vocabulary size I tested, because any byte sequence can always be represented somehow. So the usual "pick vocabulary size by UNK rate" heuristic is useless here – it can't tell the candidates apart at all – and I picked based on the compression-versus-parameter-cost trade-off instead.

Table 3: Vocabulary ablation on the test set, using a ∼10M-word training subset

	Telugu		Nepali	
Vocab	B/tok	Tok/doc	B/tok	Tok/doc
8,000	10.15	582.40	11.05	470.69
10,000	10.62	556.68	11.54	450.77
12,000	**10.99**	**537.61**	**11.90**	**436.79**
16,000	11.57	510.77	12.45	417.57
32,000	12.87	459.17	13.55	383.68

Using a larger vocabulary gives some improvement in compression, but the benefit starts to decrease while the embedding table becomes more expensive. With a 25M-parameter budget, this becomes an important limitation. For example, with $d_{\text{model}} = 512$, using a vocabulary of 32,000 tokens would use a significant part of the available parameters just for the embeddings. So, I chose a vocabulary size of **12,000** for both languages. This gives a good balance between compression and the number of parameters available for the attention and feed-forward layers. In practice, both tokenizers use around 98.7–98.8% of their available vocabulary.

Telugu shows consistently lower bytes-per-token than Nepali at every size, which means it needs more tokens to encode the same amount of text. That's a real morphological difference: Telugu is more strongly agglutinative, so a fixed-size BPE vocabulary keeps falling back to smaller sub-word pieces. Telugu's most frequent entries skew toward short case markers and suffixes (లో, ను, లు), while Nepali's include more standalone postpositions (को, मा).

3 Phase 2: Architecture and Pre-training

3.1 Implementation

The model is a decoder-only Transformer built only from `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout`. No `nn.Transformer*`, no HuggingFace, no pre-built attention. Token embeddings are summed with learned absolute positional embeddings (a 512×512 table) and passed through six Pre-LN blocks. Multi-head causal self-attention uses one fused QKV projection reshaped into 8 heads of dimension 64, with scaled dot-product attention and an additive upper-triangular

causal mask.

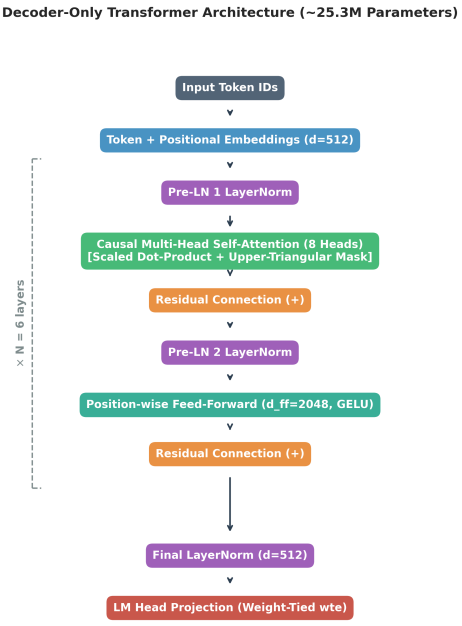


Figure 2: The decoder-only Transformer block, repeated 6 times. Every component here is built from primitive PyTorch layers – no `nn.Transformer*`, no HuggingFace.

I chose Pre-LN over Post-LN because it keeps the residual path clear through the whole stack, which trains more stably at this depth without a long, fragile warmup phase. The output head's weight is tied to the input embedding, which saves 6.14M parameters per model and pushes the input and output token representations into the same space.

Table 4: Shared configuration. Both models are architecturally identical; only their separately trained vocabularies and weights differ.

Hyperparameter	Value
d_{model} / layers / heads	512 / 6 / 8
Head dimension d_k	64
d_{ff}	2048
Context length	512
Dropout	0.1
Vocabulary	12,000
Weight tying	Yes
Optimizer	AdamW
(β_1, β_2)	(0.9, 0.95)
Max learning rate	6×10^{-4}
Schedule	warmup + cosine
Weight decay	0.1 (2-D params)
Gradient clipping	1.0
Batch size	32
Training steps	30,500
Token budget	$\approx$ 499.7M
Parameters	**25,321,472**

Six layers felt like the right middle ground at this budget. Shallower and wider would spend capacity on feed-forward width without the depth Phase 3's multi-hop reasoning would end up needing; deeper and narrower shrinks d_k past the point where each head has a useful subspace to work with. At batch 32 and context 512, each step covers 16,384 tokens, so 30,500 steps comes out to about 499.7M tokens. That means neither model finished even one full epoch over its own training split, both runs were cut off by the token budget, not by convergence.

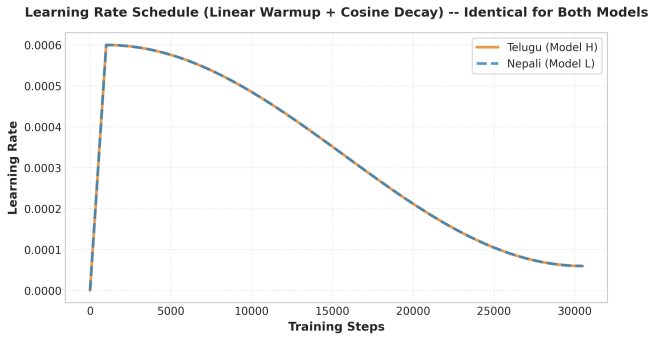


Figure 3: The learning-rate schedule, identical for both models by design: linear warmup over the first 1,000 steps, then cosine decay to 0.1× the peak rate

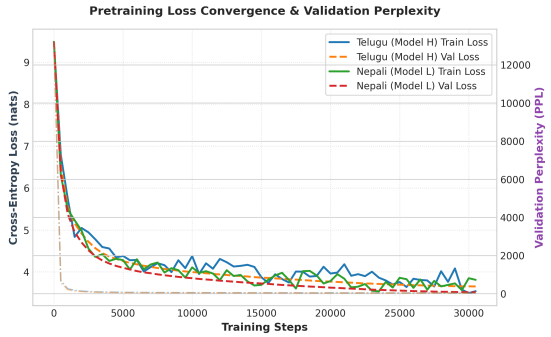
3.2 Pretraining results

Table 5: Intrinsic language-modeling metrics at step 30,500, on splits never seen during training.

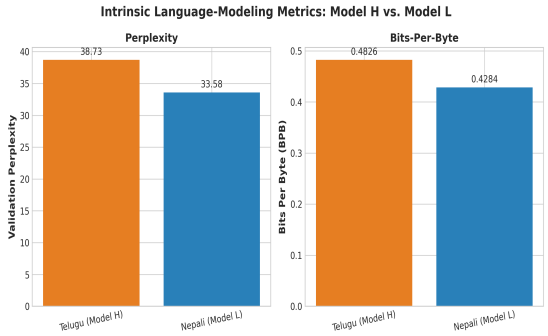
	Validation		Test	
	H	L	H	L
Loss (nats)	3.657	3.514	3.656	3.499
Perplexity	38.73	33.58	38.70	33.09
Bits-per-byte	0.483	0.428	0.483	0.426

The validation and test results are little close for both languages, which was a useful sanity check that the validation split, which I monitored during training, was not being heavily overfit. What surprised me more was that Model L performed better than Model H on all the intrinsic evaluation metrics. This was not what I expected at the beginning, since I assumed that the higher-resource language would have an advantage.

These reference-based scores in Figure 5 need to be interpreted with some caution. BLEU-4 and ROUGE-L are mainly designed for tasks such as translation and summarization, where matching the reference wording is important. For open-ended text generation, this is not always the case. A continuation can be fluent and make sense even if it is different from the reference, but it may still receive a very low score. chrF++ works with character n-grams and is a little more tolerant of morphological differences, so I find Nepali's advantage in chrF++ and ROUGE-L more useful to look at than its smaller advantage in BLEU-4. However, none of these metrics directly tell us whether the generated text is coherent, which is why I also looked at repetition and diversity mea-



(a) Training and validation loss, both models



(b) Final perplexity and bits-per-byte

Figure 4: Pretraining convergence and final intrinsic metrics. Both models show a steep drop over the first ∼10,000 steps and a long flattening tail that was still descending when the token budget ran out.

sures.Qualitatively, both models write fluently, grammatically sound text with no code-switching, though greedy decoding at this size does produce visible repetition loops.

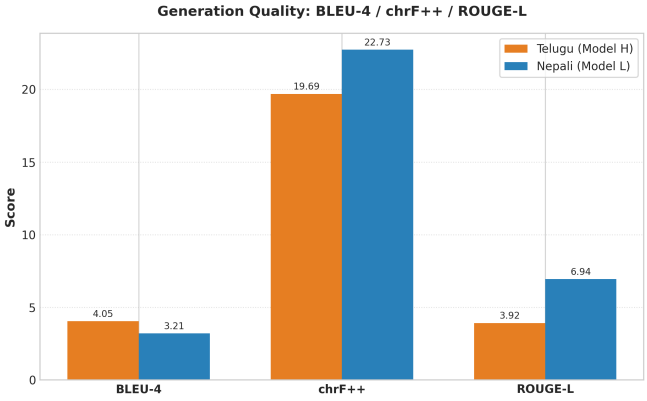


Figure 5: Reference-overlap scores for generated continuations, held-out test prefixes

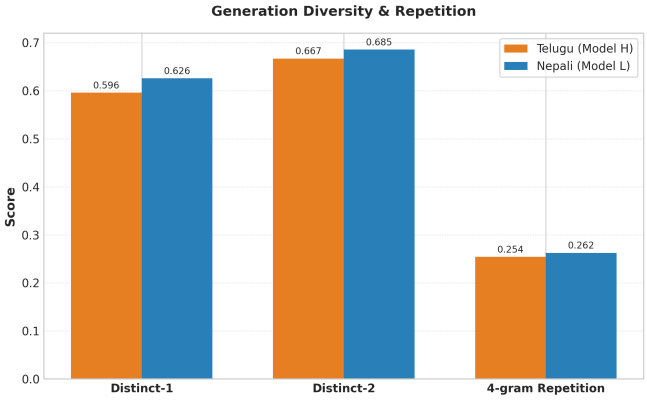


Figure 6: Distinct-1/2 and 4-gram repetition rate

3.3 Attention analysis

Since I have implemented attention from scratch, I wanted to check what it learned rather than just assume the mechanism was doing something sensible. Per-head entropy (higher means more diffuse) and mean attention distance (higher means more long-range) were computed for every layer and every head.

Table 6: Notable heads from the full entropy and distance matrices (1-indexed).

Model	Behavior	Ent.	Dist.
Telugu	Focused + local (L5,H4)	0.044	0.86
Telugu	Longest-range (L6,H3)	—	2.92
Telugu	Most diffuse (L1,H6)	1.151	—
Nepali	Most local (L4,H5)	—	0.91
Nepali	Focused + long (L5,H4)	0.125	2.87
Nepali	Most diffuse (L1,H5)	1.167	—

The sharpest Telugu attention head is highly local, mostly attending to the previous token, while the corresponding Nepali head focuses on a specific token several positions back. This suggests a content-based rather than purely positional pattern. In both models, the most diffuse heads appear in Layer 1, suggesting that early layers perform broader aggregation before becoming more specialized.

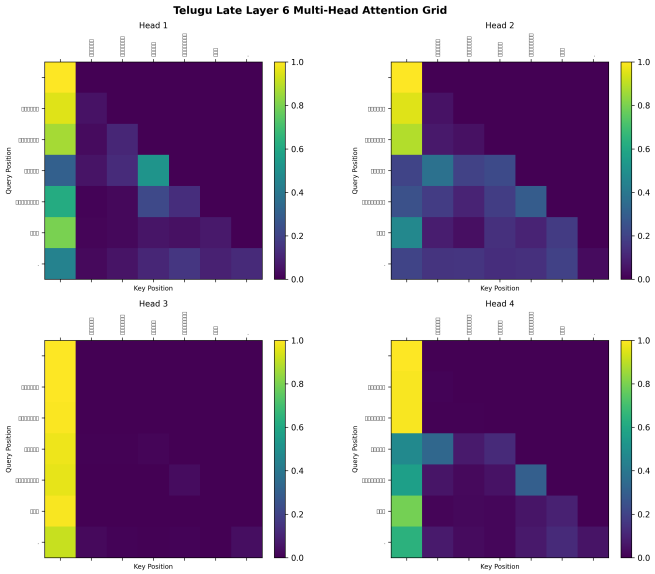


Figure 8: Four Telugu Layer-6 heads on the same sentence

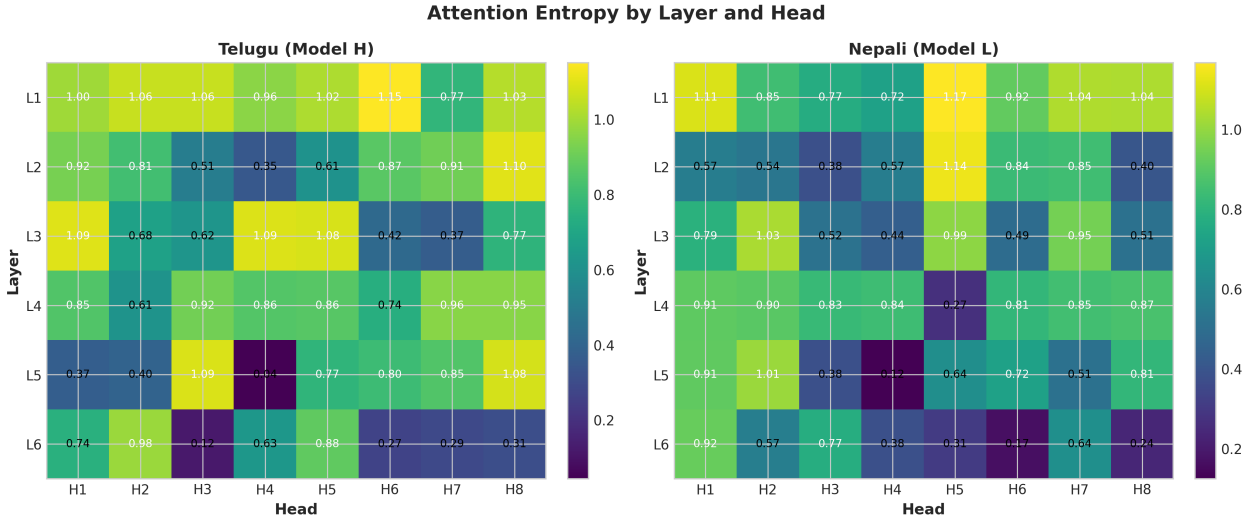


Figure 7: Attention entropy for every layer × head, both models. Darker cells are sharper heads. Both models' sharpest cells cluster in the late layers (L5–L6), and both put their most diffuse cells in L1 – broad low-level aggregation before anything specializes.

3.4 Bonus ablation: no positional embeddings

I have retrained Model L completely from scratch with positional embeddings removed, every other setting kept identical. Validation perplexity rose from 33.58 to 36.48 and BPB from 0.428 to 0.439; BLEU-4/chrF++/ROUGE-L fell from 3.21/22.73/6.94 to 2.28/21.82/5.76. The more interesting effect shows up in attention: Layer 1 Head 1 entropy roughly halved (1.108 to 0.546) and mean distance dropped by about 3× (1.929 to 0.562). Attention actually got *more* localized without position information, not more diffuse as I half-expected, once a head can no longer tell near from far, it seems to fall back on just matching nearby repeated content.

4 Phase 3: Reasoning Finetuning

4.1 Dataset and protocol

Each language has its own programmatically generated comparative-reasoning corpus; nothing is downloaded. There are four template families:

- **pairwise** — two entities, one numeric attribute, which is greater or smaller.
- **threeway** — three entities, one attribute, maximum or minimum.
- **transitive** — two chained inequalities with a question about the endpoints.
- **transitive_4hop** — a three-hop chain over four entities, generated test-only as a held-out relation pattern.

Entity names are split into disjoint train/val/test pools before any example is generated, and every template carries several statement and question phrasings chosen at random, so the task cannot be solved by memorizing one sentence shape. Splits are 4,499 / 733 / 1,147 for Telugu and 4,497 / 735 / 1,149 for Nepali.

Each model starts from its own Phase 2 checkpoint with its tokenizer and vocabulary fixed. Examples are tokenized as `[BOS]+prompt+answer+[EOS]` with all label positions before the answer span masked to `ignore_index`, so gradients come only from predicting the answer and never from reproducing the prompt.

4.2 Catastrophic forgetting

My first finetuning configuration looked totally fine on the reasoning metric, while quietly wrecking the model's general language ability. Perplexity on the Phase 1 validation split – ordinary text, nothing to do with the reasoning task – rose from 38.7 to 140.9, roughly a 264% increase. Reasoning accuracy alone couldn't have caught this, because the two are measured on completely different distributions: the reasoning task's outputs come from a tiny closed set of entity names, so a model can get very good at emitting one of two names while getting much worse at everything else in the language.

Three changes ended up fixing it:

- **Learning rate** $3 \times 10^{-5} \rightarrow 1 \times 10^{-5}$, which mattered the most. Large updates on a 4,500-example dataset with such a narrow output distribution can move the pretrained weights a surprising amount within just a few hundred steps.
- **Dropout** $0.1 \rightarrow 0.15$**, then reverted.** I tried this as extra regularization, and it actually made things worse: validation accuracy rose, peaked, then *declined*, which isn't what a healthy regularized run is supposed to look like.
- **Early stopping on validation accuracy, not loss.** Loss kept climbing even at checkpoints where accuracy was flat. That turned out to be a confidence artifact

– cross-entropy penalizes probability mass that isn't sufficiently peaked – and not a real drop in capability, so I stopped trusting the loss curve for this decision.

Table 7: General-LM perplexity against reasoning accuracy across finetuning checkpoints. Step 0 is the Phase 2 pretrained checkpoint.

Step	Gen. PPL	Δ%	Reas. acc.
Telugu (Model H)			
0	38.73	—	—
300	40.15	+3.7%	76.0%
600	**41.06**	**+6.0%**	**82.5%**
900	42.02	+8.5%	79.4%
1,200	43.08	+11.2%	78.2%
2,100	45.53	+17.6%	77.8%
Nepali (Model L)			
0	33.58	—	—
300	34.58	+3.0%	77.4%
600	**35.30**	**+5.1%**	**78.0%**
900	36.12	+7.5%	77.0%
1,200	36.85	+9.7%	76.2%
2,100	38.49	+14.6%	76.1%

Accuracy peaks at step 600 in both languages and doesn't improve after that; perplexity keeps getting worse and never recovers. Since no later step buys extra reasoning ability at a price worth paying, step 600 is the obvious stopping point – Telugu at +6.0% and Nepali at +5.1% general-LM degradation, a world away from the +264% my first attempt reached.

4.3 Curriculum template rebalancing

Uniform sampling over the three training templates just doesn't work here, and the reason is a difficulty imbalance rather than a count imbalance. `transitive` gets solved almost immediately, so most of the gradient budget ends up going toward re-teaching something the model already knows. Across every uniform-sampling run I tried, Telugu `pairwise` accuracy sat somewhere between 4% and 12% and just wouldn't budge, whatever I threw at it: lower learning rate, more data, longer training. That refusal to respond was the clue that the problem was in the sampling distribution, not in optimization.

The fix was to weight the sampler 3:1 toward the easy template early on, then interpolate linearly to 3:1 toward the hard templates by the end. Once `transitive` is solved, its examples contribute almost no loss and therefore almost no gradient, so uniform sampling was effectively wasting most of every batch. Shifting the mixture concentrates the signal where the model is still getting things wrong. Telugu `pairwise` went from that 4–12% band up to **46.00%**, and `transitive` didn't regress at all (99.60% final).

Final hyperparameters for both languages: learning rate 1×10^{-5}, weight decay 0.01, dropout 0.1, batch size 16, 150-step warmup plus cosine decay, maximum 3,000 steps with early stopping on validation accuracy (patience 12), and curriculum weights $3.0 \to 1.0$ (easy) and $1.0 \to 3.0$ (hard).

4.4 Results

Table 8: Pretrained versus finetuned exact-match accuracy on the held-out test set.

	Telugu (H)		Nepali (L)	
Template	Pre	Post	Pre	Post
pairwise	0.00%	46.00%	0.00%	32.80%
threeway	0.00%	21.20%	0.00%	11.60%
transitive	0.00%	99.60%	0.00%	88.40%
4hop (held-out)	0.00%	81.95%	0.00%	68.67%
Overall	**0.00%**	**64.69%**	**0.00%**	**52.74%**

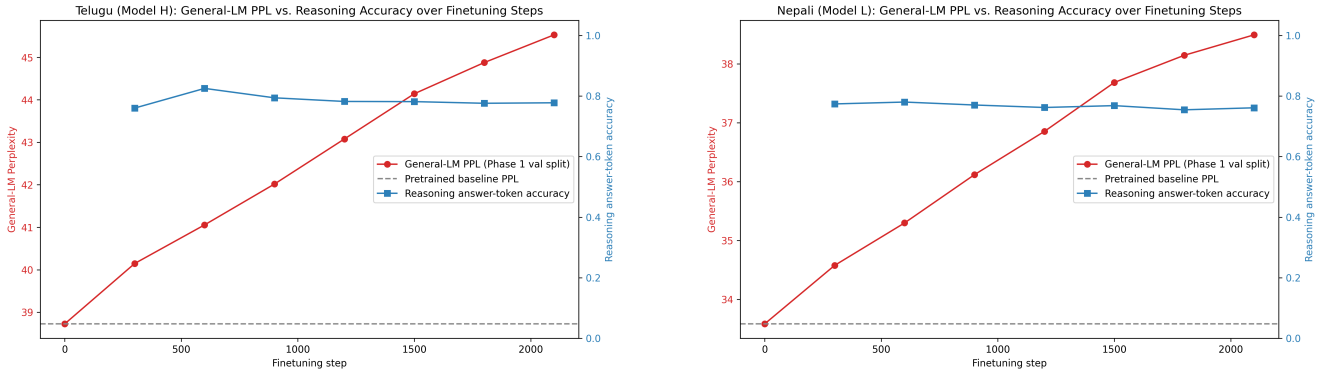
The uniform 0.00% before finetuning isn't a harness bug, it's exactly what we expect. The pretrained models are next-token predictors over news text that have never seen "సమాధానం:" ("Answer:") used as an instruction to emit one entity and stop, so naturally they just continue the passage instead. Given a weight-comparison prompt, pretrained Telugu produces "1. బరువును తగ్గించడానికి, బరువును తగ్గించడానికి..." ("1. To reduce weight, to reduce weight…"), and pretrained Nepali produces "- यो एक उचाइ हो।" ("– This is a height."). Both are fluent, both are reasonable continuations of the prompt, and neither is an answer – so exact match has no choice but to score zero.

4.5 Error analysis

The real result in Table 8 is the ordering: the two templates with *no numbers in them at all* score far above the two that need numeric comparison, in both languages.

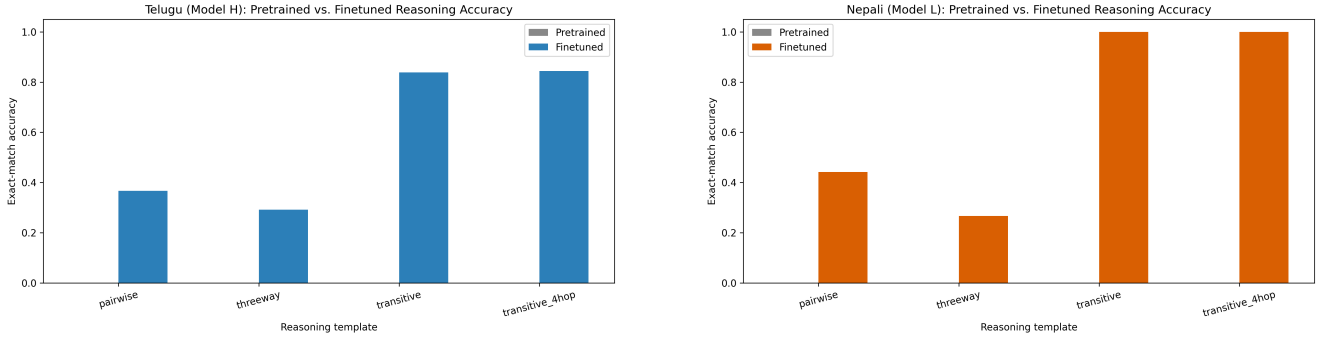
Numbers just don't tokenize with any place-value structure. BPE segments numbers by corpus frequency rather than by digit position, so the segmentation is basically inconsistent from one number to the next. Telugu encodes 146 as `[_14][6]` but 198 as a single token `[_198]`, and 49735 comes out as `[_49][7][3][5]`. Across the full range, only 2.2% of Telugu and 2.4% of Nepali three-digit numbers are single tokens. Since heights in this dataset are three-digit values, a height comparison is really asking the model to recover magnitude from fragment sequences that share no positional structure at all. `transitive` sidesteps this completely, which is presumably why it reaches 99.60% while `pairwise` reaches 46.00% in the very same model, even though transitive chaining is arguably the harder inference of the two. The gap isn't about reasoning depth. It's about whether the quantities survive tokenization in the first place.

The two languages don't fail the same way, though. Every logged Telugu `pairwise` error names an entity that actually *is* in the prompt: the model narrows it down to the right two candidates and then just picks the wrong one. That's a clean polarity error – the



(a) Telugu (Model H) **(b)** Nepali (Model L)

Figure 9: Reasoning accuracy peaks early and flattens in both languages, while general-LM perplexity keeps climbing. The gap between the two curves is the whole argument for stopping at step 600 instead of chasing a slightly higher accuracy number later.



(a) Telugu (Model H) **(b)** Nepali (Model L)

Figure 10: Exact-match accuracy by template, before and after reasoning finetuning

copying works fine, the magnitude comparison is what breaks.

Nepali shows that same error, but it also has two failure modes Telugu doesn't. The first is emitting a name that never appeared in the prompt at all: answering **गणेश** to a question about **गोविन्द** and **विमला**. Both hallucinated names live in the generator's entity pool, just not in *that* prompt, so the model seems to be pulling a plausible name from the finetuning distribution instead of actually copying from context. The second is a boundary error: the genitive postposition fuses onto the answer (**हेडफोनको** where the reference is **हेडफोन**), or the name gets truncated (**किताब** for **किताबघर**). Strict exact match scores these as wrong even though the right entity was clearly identified. Devanagari writes postpositions attached directly to the noun, so the answer boundary is genuinely fuzzier here than it is in Telugu. Some of Nepali's apparent deficit is really a scoring artifact rather than a reasoning gap, and I suspect the true difference is smaller than the 12-point gap in the table.

The held-out pattern actually generalized. `transitive_4hop` holds onto most of the two-hop accuracy (81.95% and 68.67%), even though the training set never contains a chain longer than two statements. If the transitive result were just template memorization, this number should have fallen off a cliff. It didn't, which tells me both models genuinely picked up the relation, not just the shape of the sentence.

4.6 Post-finetune attention

Table 9: Attention statistics before and after finetuning, averaged over heads and over two reasoning prompts per language.

Metric	Pretr.	Finet.	Δ
Telugu (Model H)			
Entropy, Layer 1	1.839	1.845	+0.3%
Entropy, Layer 6	1.256	1.277	+1.7%
Distance, Layer 1	6.007	5.984	−0.4%
Distance, Layer 6	6.479	6.063	**−6.4%**
Nepali (Model L)			
Entropy, Layer 1	1.992	2.001	+0.5%
Entropy, Layer 6	1.369	1.357	−0.9%
Distance, Layer 1	7.140	7.091	−0.7%
Distance, Layer 6	8.041	7.467	**−7.1%**

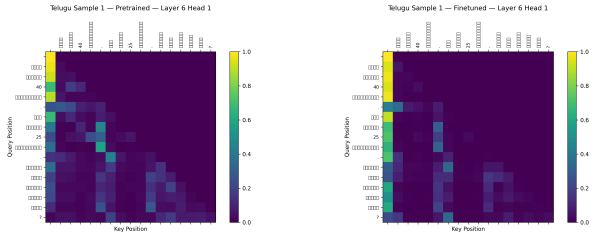


Figure 11: Telugu, Layer 6 Head 1, same reasoning prompt, before (left) and after (right) finetuning. The finetuned map pulls slightly more weight toward the diagonal

The change after finetuning is mainly seen in the later layers. Layer 1 statistics change by less than 1

The early layers appear to remain relatively stable, possibly because they handle more general language and morphological patterns, while the later layers need to adapt to the reasoning task. Both languages show a similar pattern despite using different scripts. However, since the analysis uses only two prompts per language, these results should be treated as an observation rather than a precise estimate of the overall effect.

5 Final Comparison

5.1 How did data scale and quality differ?

The corpus sizes were fairly similar, with 547.5M tokens for Telugu and 526.6M for Nepali, so both met the target. The main difference was in how the data was collected. Telugu had 28.12% of its tokens from manually collected sources, mainly because of one large news crawl, while Nepali had 20.22% spread across 34 different sources. Nepali also had a larger share of downloaded data (79.78% compared with 71.88%), making its overall corpus somewhat more dependent on those sources.

The main difference in data quality became clear from the collection process. For Nepali, the Sangraha subset alone was enough to meet the required amount of downloaded data, so the IndicCorpV2 and CC-100 fallback sources were not needed. Telugu, on the other hand, had access to a much wider range of independent web sources. This suggests that the lower-resource setting was less about having too few tokens and more about having fewer independent and varied sources of text.

5.2 How do LM and reasoning results compare?

The two models perform differently depending on the type of evaluation. Model L has better intrinsic results, with a test perplexity of 33.09 compared with 38.70 for Model H, and BPB of 0.426 compared with 0.483. It also performs better on chrF++ and ROUGE-L. In contrast, Model H performs better on the reasoning task, with an overall exact match accuracy of 64.69% compared with 52.74% for Model L, and it performs better across all the individual templates.

These results do not necessarily mean that one model is generally better than the other. Perplexity also depends on the evaluation corpus, and a more homogeneous corpus can be easier for a model to predict. Nepali has a larger share of downloaded data and its manually collected data is more concentrated around news and government sources. Telugu, in comparison, uses a wider range of independent web sources along with OCR data, making its corpus more varied and potentially harder to predict. At the same time, the Telugu model appears to transfer better to the reasoning task, which requires tracking entities and their relationships across the input. This is why looking at perplexity alone would not give a complete picture of the differences between the two models.

5.3 Which factors most affected Model L?

Three factors seem to explain most of the difference. The first is the tokenizer. Since numbers are not represented with any real place-value structure, both models struggle with numerical comparison. This is also the clearest reason for the large difference between templates within the same model: Telugu gets 99.60% accuracy on the transitive template but only 21.20% on the three-way template.

The second factor is more specific to Nepali. Devanagari often combines postpositions with the words before them, which makes the exact answer-span boundary less clear. As a result, some answers are marked wrong by exact match even when the model identifies the correct entity. This may make the measured difference between the two models look somewhat larger than it actually is.

The third factor is corpus homogeneity, although I am less certain about this one. Nepali's narrower source base could explain its lower perplexity as well as some of its weaker copy accuracy. For example, the model may learn which names are common without always learning to copy the exact name from the context. However, this is only a possible explanation. More training runs would be needed to separate this effect from normal run-to-run variation.

5.4 What evidence backs these claims?

The tokenization explanation is the one I am most confident about because I measured the trained tokenizers directly. Only 2.2–2.4% of three-digit numbers are represented as single tokens. The templates involving numbers achieve 46.00% and 21.20% accuracy, while the template without numbers reaches 99.60%, all within the same model and training run. The generalization result is also supported by the `transitive_4hop` template, which was not seen during training but still achieved 81.95% and 68.67% accuracy. For forgetting, I tracked perplexity at every checkpoint on a separate held-out split, which revealed a +264% degradation that was not obvious from reasoning accuracy alone. I also looked at the actual generated answers to understand the common error patterns instead of relying only on the overall score. Finally, the attention results show a similar pattern in both indepen-

dently trained models, despite their different scripts.

The weaker parts of the analysis are the exact size of the attention changes, since they are based on only two prompts per language, and the corpus-homogeneity explanation. The latter comes from only one training run for each model, so it is difficult to separate this effect from normal variation between runs.

6 Conclusion

Two fully independent monolingual Transformer language models got built end to end: corpora of 547M and 527M tokens collected and cleaned from scratch, with 28.12% and 20.22% manually gathered; tokenizers trained from scratch at an ablation-chosen 12,000 vocabulary; a 25.3M-parameter decoder-only Transformer built from primitive layers with empirically verified causal masking; pretrained to a 500M-token budget reaching 38.70 and 33.09 test perplexity; and finetuned on self-generated comparative-reasoning data to 64.69% and 52.74% exact match, up from a 0.00% baseline, while keeping general-language degradation to +6.0% and +5.1%. The bonus no-positional-embeddings ablation ran on Model L with the full Phase 2 evaluation suite.

All code, configs, logs, evaluation results, and figures live in the project repository, with reproduction steps and checkpoint links in the top-level `README.md` and the per-phase READMEs.